

Documentação de Manutenção de Software

Equipe Clean Air — SENAI Regional Metropolitana, Lauro de Freitas/BA

29 de maio de 2026

Sumário

1	Introdução	3
2	Visão Geral do Sistema	3
2.1	Descrição	3
2.2	Tecnologias Utilizadas	4
3	Procedimentos de Manutenção	4
3.1	Atualizações	4
3.2	Correção de Bugs	5
3.3	Monitoramento e Logs	6
4	Resolução de Problemas	6
4.1	Erros Comuns e Soluções	6
5	Histórico de Versões	7
6	Configuração e Implantação	9
6.1	Etapa 1 Banco de Dados (Neon)	9
6.1.1	Criar Conta no Neon	9
6.1.2	Criar o Projeto	9
6.1.3	Obter a DATABASE_URL	9
6.1.4	Inicialização das Tabelas	10
6.1.5	Migrações Manuais (quando aplicável)	10
6.2	Etapa 2 Backend (Render)	10
6.2.1	Fazer Fork do Repositório	10
6.2.2	Criar Conta e Projeto no Render	10
6.2.3	Configurar o Web Service	11
6.3	Etapa 3 Frontend (Vercel)	11
6.3.1	Criar Conta e Importar Projeto	11
6.3.2	Configurar o Projeto	12
6.3.3	Configurar Comandos de Inicialização	12
6.3.4	Configurar Variável de Ambiente	12
6.3.5	Finalizar a Configuração no Render	12
6.4	Etapa 4 CI/CD (GitHub Actions)	12
6.4.1	Desativar o Auto-deploy no Render	12
6.4.2	Adicionar Secrets no GitHub	13

6.4.3	Habilitar e Executar os Workflows	13
6.4.4	Configurar Branch Protection Rule	13
6.5	Implantação Concluída	14
7	CI/CD - GitHub Actions	14
7.1	Script de CI	14
7.2	Script de CD	16
7.3	Pipeline	16
8	Contato e Suporte	17

1 Introdução

Este documento descreve os procedimentos de manutenção do software **Clean Air**, sistema IoT de monitoramento de múltiplas variáveis ambientais em tempo real. Ele inclui informações sobre atualizações, solução de problemas, boas práticas de desenvolvimento, monitoramento e suporte.

O Clean Air vem com a proposta de trazer um sistema que se destaque dos demais existentes em monitoramento através de uma interface minimalista e de fácil utilização para pessoas menos proficientes em tecnologia. O sistema encontra-se disponível publicamente em <https://cleanairsenailauro.vercel.app/> e seu código-fonte está hospedado em <https://github.com/jhonxxz06/TrabalhoIoT2025.2-Senai>.

2 Visão Geral do Sistema

2.1 Descrição

O Clean Air é uma plataforma web IoT multitenant de monitoramento de variáveis ambientais em tempo real. O sistema permite que organizações (domínios) cadastrem microcontroladores equipados com sensores, configurem painéis de visualização customizados (widgets) e acompanhem as leituras recebidas via protocolo MQTT. Os dados são exibidos graficamente e podem ser exportados em formato CSV para análise posterior.

O sistema adota arquitetura MVC (Model-View-Controller) e modelo de multitenancy por domínio, isolando dados e permissões por organização. As principais funcionalidades incluem:

- Gerenciamento de domínios (multitenancy) com criação de espaços isolados por organização.
- Cadastro e autenticação de usuários com controle de acesso por papel (RBAC): Superadmin, Gerente (admin) e Usuário comum (user).
- Registro e configuração de dispositivos IoT com parâmetros MQTT (broker, porta, tópico e credenciais).
- Recepção, validação (Zod) e armazenamento de dados enviados pelos sensores via MQTT.
- Exibição de dashboards com widgets configuráveis e atualização em tempo real via WebSocket (Socket.IO).
- Consulta de histórico de leituras filtrado por período (últimas 24h ou semana).
- Consulta de excedências de thresholds configurados por widget.
- Exportação de dados históricos em formato CSV com BOM UTF-8, separador ponto-e-vírgula e compatibilidade com Microsoft Excel.
- Solicitação e aprovação de acesso de usuários a dispositivos específicos.

2.2 Tecnologias Utilizadas

- **Linguagens:** JavaScript (Node.js no backend, React 18 no frontend).
- **Banco de Dados:** PostgreSQL 16 hospedado no Neon (serverless cloud). Conexão via driver pg com pool configurado com timezone `America/Sao_Paulo`.
- **Frameworks e Bibliotecas (Backend):** Express 4, Socket.IO 4 (servidor Web-Socket), biblioteca `mqtt` v5 (cliente MQTT), Zod 3 (validação de schemas), Helmet 7 (headers de segurança HTTP), `jsonwebtoken` + `bcryptjs` (autenticação JWT).
- **Frameworks e Bibliotecas (Frontend):** React 18, Chart.js, Socket.IO Client.
- **Infraestrutura:** Frontend hospedado na Vercel; backend hospedado no Render; Banco de dados hospedado no Neon.
- **Protocolo IoT:** MQTT 5 — o backend assina automaticamente os tópicos configurados em cada dispositivo cadastrado.
- **Versionamento:** Git e GitHub, com padrão de Conventional Commits (`fix`, `feat`, `docs`, `refactor`) e estratégia de branches GitFlow adaptado (branches `main` e `cloud-database`).

3 Procedimentos de Manutenção

3.1 Atualizações

O processo de CI/CD do Clean Air é automatizado via integração entre o repositório GitHub e os serviços de hospedagem:

1. **Frontend (Vercel):** Qualquer commit ou merge realizado na branch `main` dispara automaticamente um novo build e deploy na Vercel. O link de produção permanece estável em <https://cleanairsenailauro.vercel.app/>.
2. **Backend (Render):** Da mesma forma, commits na branch `main` disparam rebuild automático do Web Service no Render. O endpoint do backend termina com `.onrender.com`.
3. **Banco de Dados (Neon):** O schema do banco é gerenciado de duas formas:
 - **Automática:** Na inicialização do servidor, a função `initDatabase()` chama `createTables()`, que executa `CREATE TABLE IF NOT EXISTS` e `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` para garantir o schema atualizado sem intervenção manual.
 - **Manual (migrações pontuais):** Alterações de schema que não fazem parte do fluxo automático ficam em arquivos `.sql` versionados no repositório (ex.: `20251215_mqtt_received_at_timestamptz.sql`) e são aplicadas com o script `run_migration.js`:

```
node run_migration.js 20251215_mqtt_received_at_timestamptz.sql
```

Procedimento de rollback: Caso um deploy cause instabilidade, o Render e a Vercel permitem redeploy de versões anteriores diretamente pelo painel. Para o banco de dados, os arquivos de rollback (ex.: `20251215_mqtt_received_at_revert_to_timestamp.sql`) são aplicados via `run_migration.js`.

Variáveis de ambiente obrigatórias (Render/Backend):

Variável	Descrição
DATABASE_URL	Connection string do Neon PostgreSQL
FRONTEND_ORIGIN	URL do frontend na Vercel (ex.: <code>https://projeto.vercel.app</code>)
JWT_SECRET	Segredo alfanumérico para assinatura dos tokens JWT
JWT_EXPIRES_IN	Tempo de expiração do token (ex.: 24h)
MQTT_BROKER	URL do broker MQTT padrão (ex.: <code>mqtt://broker.hivemq.com:1883</code>)
PORT	Porta do servidor (ex.: 3001)

Variáveis de ambiente obrigatórias (Vercel/Frontend):

Variável	Descrição
REACT_APP_API_URL	URL completa do backend no Render

3.2 Correção de Bugs

O fluxo de correção de bugs adotado pela equipe segue as etapas abaixo:

1. **Identificação:** Bugs são identificados via Render Logs (backend), Neon Console (banco de dados) ou relato de usuários pelo canal de suporte.
2. **Registro:** O bug é registrado como card no quadro Trello do projeto, classificado por severidade e atribuído a um membro da equipe.
3. **Reprodução local:** O desenvolvedor clona a branch `cloud-database`, configura as variáveis de ambiente locais e reproduz o bug em ambiente `localhost`.
4. **Correção e commit:** A correção é implementada e commitada seguindo o padrão Conventional Commits com prefixo `fix`. Exemplo: `fix: corrigir validação de payload com valores nulos`.
5. **Deploy automático:** O push / merge para a branch `main` dispara o CI/CD automaticamente no Render e na Vercel.
6. **Validação:** O bug é validado em produção e o card no Trello é movido para a coluna **Feito**.

Bugs relacionados ao banco de dados devem ser investigados prioritariamente no Neon Console, verificando logs de queries e estado das tabelas. Bugs de integração MQTT podem ser diagnosticados com o MQTT Explorer, simulando publicações de payload para o broker configurado.

3.3 Monitoramento e Logs

O Clean Air utiliza os recursos de monitoramento nativos dos serviços de hospedagem:

- **Render Logs:** Principal ferramenta para monitoramento do backend. Exibe em tempo real logs de conexões MQTT, erros de validação de payload, erros de banco de dados e exceções não tratadas. Acessível em: Painel Render → Web Service → aba *Logs*.
- **Neon Console:** Permite inspecionar o estado das tabelas, executar queries SQL diretamente e monitorar o crescimento do banco de dados. Acessível em: <https://console.neon.tech>.
- **Chrome DevTools:** Utilizado nos testes manuais do frontend para inspecionar requisições HTTP, eventos WebSocket e erros de JavaScript no console do navegador.
- **MQTT Explorer:** Ferramenta desktop utilizada para simular publicações de payload MQTT nos tópicos configurados, facilitando diagnóstico de problemas na cadeia de recepção de dados.
- **Vercel Analytics:** Disponível no painel da Vercel para monitoramento de disponibilidade e performance do frontend.

Itens do backlog relacionados ao monitoramento:

- **B008** (Prioridade Média): Implementação de logs estruturados com Winston ou Pino para facilitar diagnóstico em produção.
- **B007** (Prioridade Alta): Implementação de testes unitários e de integração automatizados.

4 Resolução de Problemas

4.1 Erros Comuns e Soluções

- **Backend não inicia / erro de conexão com o banco:** Verificar se a variável `DATABASE_URL` está corretamente configurada nas variáveis de ambiente do Render. Confirmar que o projeto Neon está ativo e que a connection string inclui o parâmetro `?sslmode=require`.
Formato esperado:
`postgres://usuario:senha@ep-exemplo.us-east-2.aws.neon.tech/neondb?sslmode=require`.
- **CORS bloqueando requisições do frontend:** Verificar se a variável `FRONTEND_ORIGIN` no Render contém exatamente a URL do frontend na Vercel, incluindo o protocolo `https://` e sem barra final. Exemplo: `https://cleanairsenailauro.vercel.app`.
- **Payload MQTT rejeitado (não persiste no banco):** O sistema rejeita payloads onde qualquer valor não seja numérico. Verificar o firmware do microcontrolador e garantir que o JSON publicado segue o formato `{"campo": valor_numerico}`. Exemplo válido: `{"temperature": 25.3, "humidity": 61.2}`. Payloads rejeitados são registrados nos Render Logs com o prefixo `[MQTT] Payload rejeitado`.

- **Dashboard não atualiza em tempo real:** Verificar se o frontend consegue estabelecer conexão WebSocket com o backend. Inspecionar no Chrome DevTools (aba Network → WS) se a conexão Socket.IO está ativa. Causas comuns: variável `REACT_APP_API_URL` apontando para URL incorreta, ou plano do Render não suportando conexões persistentes.
- **Usuário redirecionado para tela de espera após aprovação:** O frontend faz polling a cada 10 segundos para verificar o flag `has_access`. Verificar no Neon Console se o campo `has_access` do usuário foi de fato atualizado para 1 na tabela `users` após a aprovação pelo gerente.
- **Erro 401 Unauthorized em rotas protegidas:** O token JWT expirou ou não foi enviado no cabeçalho `Authorization: Bearer {token}`. O usuário deve realizar login novamente. Verificar se `JWT_EXPIRES_IN` está configurado corretamente (recomendado: 24h).
- **Erro 403 Forbidden ao tentar criar/editar dispositivo:** O usuário autenticado possui `role='user'` e está tentando acessar uma rota restrita a `role='admin'`. Verificar se o login foi realizado com a conta de gerente correta.
- **Dispositivo MQTT não conecta ao broker:** Verificar se o endereço do broker, porta, tópico e credenciais (quando necessárias) foram configurados corretamente no cadastro do dispositivo. Usar o MQTT Explorer para testar a conexão diretamente com o broker antes de cadastrá-lo no sistema. Brokers públicos como `broker.hivemq.com` podem apresentar instabilidade; recomenda-se o uso de broker privado com autenticação para ambientes de produção.
- **Arquivo CSV exportado com caracteres corrompidos no Excel:** O arquivo é gerado com BOM UTF-8, que deve ser reconhecido automaticamente pelo Excel. Caso os caracteres apareçam corrompidos, abrir o Excel, usar *Dados → De Texto/CSV* e selecionar codificação UTF-8 manualmente.
- **Erro nas migrações manuais:** Ao executar `node run_migration.js <arquivo.sql>`, garantir que a variável `DATABASE_URL` está definida no ambiente local ou que o comando é executado via Render Shell com a variável configurada. Migrações manuais só são necessárias em bancos com dados existentes; em implantações novas o `createTables()` já aplica o schema correto.

5 Histórico de Versões

Versão	Data	Alterações	Responsável
1.1.2	06/12/2025	Inserção de um novo widget (tabela de exceções); Correção quanto a organização e preenchimento dos dados das planilhas; Aumento de tamanho dos widgets	João Pedro
1.2.0	06/12/2025	Inserção do protocolo Websocket	Erick Reis Santos

Versão	Data	Alterações	Responsável
1.2.1	10/12/2025	Alteração voltada para dashboards não ultrapassassem o traçado da whiteboard	Erick Reis Santos
1.2.2	10/12/2025	Alteração para que o layout organizado pelo administrador fosse o mesmo visualizado pelo usuário	João Pedro Santana
1.3.0	14/12/2025	Deploy do projeto para serviços em nuvem (render, vercel e Neon)	João Pedro Santana, Erick Reis Santos
1.3.1	15/12/2025	Corrigir falha de concordância entre os dados de horário salvos no banco de dados com os dados reais	João Pedro Santana
1.4.0	08/04/2026	Substituição dos alerts.js para notificações com elementos toasts em react.js; Remoção de informações sobre rotas de api, validações e requisições do console log	João Pedro Santana
1.4.1	16/04/2026	Adição de uma verificação quanto ao formato do payload recebido com o suporte do dashboard, para evitar eventuais quebras do mesmo ao receber formatos inesperados	João Pedro Santana
1.5.0	22/04/2026	Implementação da arquitetura multi tenant	João Pedro Santana
1.5.1	23/04/2026	Corrigir problema em que usuários pertencentes de outros domínios estavam sendo listados em domínios aos quais não pertenciam	João Pedro Santana
1.5.2	24/04/2026	Correção de falha no fluxo de permissões em que o dispositivo não era exibido ao usuário caso seu acesso fosse recusado previamente e, posteriormente, concedido de forma manual por um Administrador	João Pedro Santana
1.6.0	11/05/2026	Inserção do script de CI, atualizações estéticas (novos toasts.js), Inserção de campos para atualização dos perfis	João Pedro Santana
1.7.0	14/05/2026	Inserção do script de CD e funcionalidade de forçar lowercase nos campos de email dos cadastros	João Pedro Santana

Versão	Data	Alterações	Responsável
1.7.1	15/05/2026	Edição do script de CI para executar com pull requests e edição do script de CD para realizar o deploy somente do render	João Pedro Santana

6 Configuração e Implantação

6.1 Etapa 1 Banco de Dados (Neon)

A configuração do banco de dados no Neon é um pré-requisito para as etapas seguintes. Após concluída, você deverá ter em mãos a `DATABASE_URL` que será utilizada nas variáveis de ambiente do backend.

6.1.1 Criar Conta no Neon

1. Acesse `neon.tech` e clique em *Sign up*. Crie sua conta utilizando GitHub, autorizando o acesso do Neon à sua conta.

6.1.2 Criar o Projeto

2. Preencha os campos de configuração do projeto conforme abaixo:
 - **Project name:** nome de sua preferência (ex.: `clean-air`)
 - **Postgres version:** 16
 - **Region:** selecione a região mais próxima da sua aplicação (recomendado: AWS South America East 1)
3. Clique em *Create project* para confirmar. Em seguida, clique em *Go to project*.

6.1.3 Obter a DATABASE_URL

4. No painel do projeto recém-criado, clique no botão *Connect*.
5. Na janela que abrir, copie a connection string clicando no botão *Copy snippet* — ela será a sua `DATABASE_URL`. Guarde-a com segurança, pois será utilizada como variável de ambiente no Render.

Nota: Exemplo de `DATABASE_URL`:

```
postgresql://usuario:senha@ep-exemplo-123456.us-east-2.aws.neon.tech/neondb?sslmode=require
```

6.1.4 Inicialização das Tabelas

Não é necessária nenhuma ação manual para criar as tabelas. Ao iniciar pela primeira vez no Render, o backend executa automaticamente `initDatabase()` (`index.js`), que chama `createTables()` (`database.js`), criando todas as tabelas via `CREATE TABLE IF NOT EXISTS`.

Para que isso funcione, basta garantir que a `DATABASE_URL` esteja corretamente configurada no Render conforme a Etapa 2.

6.1.5 Migrações Manuais (quando aplicável)

Alterações pontuais de schema ficam em arquivos `.sql` versionados no repositório, aplicados via script `run_migration.js`. Exemplos de migrações presentes no repositório:

- `20251215_mqtt_received_at_timestamptz.sql` — converte `received_at` para `TIMESTAMP WITH TIME ZONE`
- `20251215_mqtt_received_at_revert_to_timestamp.sql` — reverte para `TIMESTAMP` simples (rollback)

Para aplicar uma migração, execute o comando abaixo na raiz do backend (localmente ou via Render Shell), com a `DATABASE_URL` definida no ambiente:

```
node run_migration.js <nome_do_arquivo.sql>
```

Exemplo: `node run_migration.js 20251215_mqtt_received_at_timestamptz.sql`

Nota: As migrações manuais só precisam ser executadas em implantações que já possuem dados na coluna afetada. Em uma implantação nova (banco vazio), o schema correto já é aplicado automaticamente por `createTables()`.

6.2 Etapa 2 Backend (Render)

O backend da aplicação será hospedado no Render. Antes de iniciar, é necessário que o repositório esteja na sua conta do GitHub via Fork.

6.2.1 Fazer Fork do Repositório

6. Acesse o repositório original no GitHub: github.com/jhonxxz06/TrabalhoIoTds2025.2-Senai
7. Clique no botão *Fork* no canto superior direito da página.
8. Preencha o campo *Repository name* com o nome de sua preferência.
9. Clique no botão *Create Fork* para confirmar.

6.2.2 Criar Conta e Projeto no Render

10. Acesse o Render em: render.com
11. Clique em *Start for free* e crie sua conta utilizando login via GitHub (isso facilita o acesso ao repositório forkado), autorizando acesso de terceiros à sua conta.
12. No campo *Create a new service*, selecione a opção *Web Services*.

6.2.3 Configurar o Web Service

13. Na tela de seleção de fonte de código, escolha a opção *Git Provider*.
14. Selecione o GitHub como provedor.
15. Marque a opção *Only select repositories* e selecione o repositório fork criado.
16. Clique em *Install* e, em seguida, selecione o repositório.
17. Preencha os campos de configuração conforme abaixo:
 - **Name:** nome de sua escolha
 - **Language:** NODE
 - **Branch:** main
 - **Root Directory:** backend
 - **Build Command:** npm install
 - **Start Command:** npm run dev
 - **Instance Type:** de acordo com a necessidade do projeto
18. Preencha as variáveis de ambiente conforme a tabela abaixo (deixe o Caps Lock ativo ao digitar os nomes):

NAME_OF_VARIABLE	VALUE
DATABASE_URL	URL obtida no Neon (etapa 6.1.3)
FRONTEND_ORIGIN	deixar em branco provisoriamente
JWT_EXPIRES_IN	24h
JWT_SECRET	código alfanumérico de sua preferência
MQTT_BROKER	mqtt://broker.hivemq.com:1883
PORT	3001

Nota: Exemplo de JWT_SECRET: J9.eyJrZXkiOiJ2YWx1 (código alfanumérico gerado por você).

19. Pressione *Deploy Web Service* para iniciar o deploy. Na aba *Events*, copie o link exibido (ele termina com `.onrender.com`). Guarde este link para uso nas etapas seguintes.

6.3 Etapa 3 Frontend (Vercel)

O frontend será hospedado na Vercel. Certifique-se de ter o link do backend (Render) em mãos antes de iniciar esta etapa.

6.3.1 Criar Conta e Importar Projeto

20. Acesse vercel.com e crie uma conta utilizando sua conta do GitHub.
21. Na tela de *new project*, vá em *Import Git Repository* e clique na opção do GitHub.
22. Selecione o repositório fork.

6.3.2 Configurar o Projeto

23. Após selecionar o repositório, configure o projeto preenchendo os seguintes pontos:

- **Project Name:** nome de sua preferência
- **Application Preset:** Create React App
- **Root Directory:** clique em *Edit* e selecione **Frontend > teste-mcp**

6.3.3 Configurar Comandos de Inicialização

24. Pressione *Build and Output Settings* e preencha os seguintes campos:

- **Build Command:** deixe ativo e insira `npm run build`
- **Output Directory:** deixe ativo e insira `build`

6.3.4 Configurar Variável de Ambiente

25. Pressione *Environment Variables* e preencha conforme abaixo:

Key	Value
REACT_APP_API_URL	link copiado do Render (etapa 6.2.3)

26. Clique em *Deploy* e aguarde.

27. Ao concluir, pressione *Continue to Dashboard*.

28. Na tela de *Overview*, copie o link que está abaixo de *Domains*.

6.3.5 Finalizar a Configuração no Render

29. Retorne ao Render e acesse as variáveis de ambiente do seu Web Service.

30. Preencha o campo `FRONTEND_ORIGIN` com o link do Domain copiado da Vercel, precedido de `https://`:

KEY	VALUE
FRONTEND_ORIGIN	https://seu-projeto.vercel.app

Nota: Exemplo: `https://clean-air.vercel.app`

6.4 Etapa 4 CI/CD (GitHub Actions)

Esta etapa configura os pipelines de integração e entrega contínua. Certifique-se de ter em mãos todos os valores das variáveis de ambiente das etapas anteriores antes de iniciar.

6.4.1 Desativar o Auto-deploy no Render

31. Ainda no Render, acesse o campo *Settings*.

32. No campo *Deploy*, desative a opção *Auto-deploy* e salve a mudança.

33. No mesmo campo, copie o *Deploy Hook* e guarde-o para uso na etapa 6.4.2.

6.4.2 Adicionar Secrets no GitHub

34. Acesse o repositório forkado no GitHub e clique em *Settings*.
35. Acesse *Secrets and variables > Actions*.
36. Na aba *Secrets*, clique em *New repository secret* e adicione cada item da tabela abaixo. A cada novo secret, pressione *Add secret* antes de prosseguir para o próximo.

KEY	VALUE
DATABASE_URL	URL obtida no Neon (etapa 6.1.3)
FRONTEND_ORIGIN	link do Domain da Vercel (etapa 6.3.5)
JWT_EXPIRES_IN	24h
JWT_SECRET	o mesmo código utilizado no Render (etapa 6.2.3)
MQTT_BROKER	mqtt://broker.hivemq.com:1883
PORT	3001
REACT_APP_API_URL	link do backend no Render (etapa 6.2.3)
RENDER_DEPLOY_HOOK	Deploy Hook copiado do Render (etapa 6.4.1)

Nota: Os nomes dos secrets devem ser escritos exatamente como na coluna **KEY** (com Caps Lock ativo). O campo *Name* corresponde à coluna **KEY** e o campo *Secret* corresponde à coluna **VALUE**.

6.4.3 Habilitar e Executar os Workflows

37. Acesse o campo *Actions* próximo ao topo da tela do repositório.
38. Marque a opção *I understand my workflows, go ahead and enable them*.
39. Selecione o workflow *CI Clean Air*.
40. Pressione o botão *Run workflow* nas duas vezes que aparecer.
41. Aguarde a execução ser concluída com sucesso antes de prosseguir.

6.4.4 Configurar Branch Protection Rule

42. Retorne a *Settings* e acesse o campo *Branches*.
43. Clique em *Add branch ruleset*.
44. Defina o nome da ruleset (ex.: **main-protection**).
45. Em *Enforcement status*, selecione *Active*.
46. No campo *Branch targeting criteria*, clique em *Add target*, selecione *Include by pattern*, digite **main** e confirme.
47. No campo *Branch rules*, marque as seguintes opções:
 - *Restrict deletions*

- *Require a pull request before merging* e preencha:
 - Required approvals: 1
 - Dismiss stale pull request approvals when new commits are pushed
 - Require approval of the most recent reviewable push
 - Require conversation resolution before merging
- *Require status checks to pass* e preencha:
 - Require branches to be up to date before merging
 - Em *Status checks that are required*, clique em *Add checks*, digite **CI - Backend** e selecione a opção do GitHub Actions
 - Repita o passo acima para **CI - Frontend**
- *Block force pushes*

48. Após marcar todas as opções, clique em *Create* para salvar a ruleset.

6.5 Implantação Concluída

Finalizada a etapa 6.4.4, a implantação do sistema Clean Air está completa. O frontend estará acessível pelo link do Domain da Vercel, a comunicação com o backend estará configurada, o banco de dados em operação, assim como os scripts de CI/CD no GitHub Actions.

7 CI/CD - GitHub Actions

7.1 Script de CI

```
name: CI Clean Air

on:
  push:
    branches:
      - cloud-database

  pull_request:
    branches:
      - main

  workflow_dispatch:

jobs:
  CI-FRONTEND:
    name: CI - Frontend
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: ./frontend/teste-mcp

    steps:
      - name: Checkout do código
```

```

    uses: actions/checkout@v4

- name: Configurar Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '22'
    cache: 'npm'
    cache-dependency-path: frontend/teste-mcp/package-lock.json

- name: Instalar dependências
  run: npm ci

- name: Build do frontend
  run: npm run build
  env:
    CI: false
    REACT_APP_API_URL: ${ secrets.REACT_APP_API_URL }

CI-BACKEND:
  name: CI - Backend
  runs-on: ubuntu-latest

  defaults:
    run:
      working-directory: ./backend

  steps:
    - name: Checkout do código
      uses: actions/checkout@v4

    - name: Configurar Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '22'
        cache: 'npm'
        cache-dependency-path: backend/package-lock.json

    - name: Instalar dependências
      run: npm ci

    - name: Subir backend e validar health
      run: |
        node src/index.js &
        sleep 5
        curl --fail http://localhost:${ secrets.PORT }/api/health
      env:
        PORT: ${ secrets.PORT }
        DATABASE_URL: ${ secrets.DATABASE_URL }
        JWT_SECRET: ${ secrets.JWT_SECRET }
        FRONTEND_ORIGIN: ${ secrets.FRONTEND_ORIGIN }
        JWT_EXPIRES_IN: ${ secrets.JWT_EXPIRES_IN }
        MQTT_BROKER: ${ secrets.MQTT_BROKER }

CI-STATUS:
  name: CI - Status Final
  runs-on: ubuntu-latest
  needs:
    - CI-FRONTEND
    - CI-BACKEND

```

```

steps:
  - name: OK
    run: echo "CI aprovada com sucesso"

```

7.2 Script de CD

```

name: CD Clean Air

on:
  push:
    branches:
      - main

jobs:
  DEPLOY:
    name: Deploy Backend (Render)
    runs-on: ubuntu-latest

    steps:
      - name: Deploy no Render
        run: curl -X POST ${ secrets.RENDER_DEPLOY_HOOK }

```

7.3 Pipeline

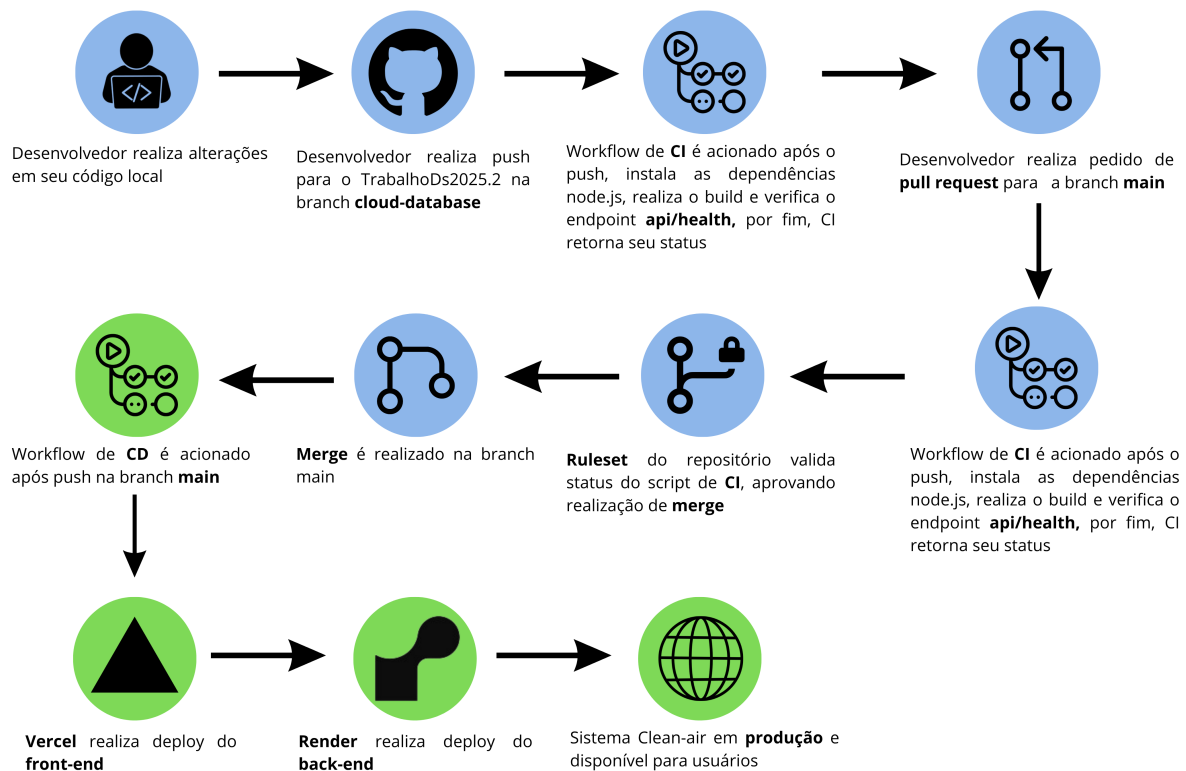


Figura 1: Pipeline CI/CD

8 Contato e Suporte

- **Canal de suporte:** E-mail cleanair.suporte@gmail.com
- **Repositório do projeto:** <https://github.com/jhonxxz06/TrabalhoIotDs2025.2-Senai>
- **Sistema em produção:** <https://cleanairsenailauro.vercel.app/>
- **Orientador:** Prof. Caio Luiz Mota Barreto — SENAI Regional Metropolitana, Lauro de Freitas/BA.
- **Equipe de Desenvolvimento:**
 - João Pedro Santana de Oliveira Menezes (021.859853)
 - Erick Reis Santos de Jesus (232.984958)
 - Wendell Trindade dos Santos Reis (232.975949)
 - Luan Gomes Correia (232.983597)
- **Instituição:** SENAI Regional Metropolitana — Lauro de Freitas/Bahia — Curso Técnico em Desenvolvimento de Sistemas.